

GitHub Progressive Pull Request Playbook

How to write, push, review, and merge the 7 PRs on GitHub

True Single-Theme Pull Request Playbook — μ OS++ IIIe SMP Integration

- 1. Architectural Principle: Pure Single-Theme Pull Requests
- 2. Catalog of Pure Single-Theme Pull Requests
- 3. Self-Contained Compilation & Testing Verification Matrix
- 4. Command-Line Rebase & Stacked PR Maintenance Cheatsheet
- 5. Web Interface Guide: Generating and Managing PRs on GitHub.com

True Single-Theme Pull Request Playbook — μOS++ IIIe SMP Integration

Exclusively Single-Subject Pull Requests on GitHub (github.com/dan-maio → github.com/micro-os-plus)

1. Architectural Principle: Pure Single-Theme Pull Requests

In the initial migration scratchpad, early fixes were artificially bundled under generic buckets (e.g. "Step 1" lumped together POSIX `dirent.h` empty structs, C++ template list iterators in `lists.h`, newlib syscall macros, and exported thread suspend symbols).

These changes are completely unrelated. A maintainer reviewing C++ intrusive list iterators should not have to review POSIX directory structs or newlib syscall aliases.

This playbook enforces a **pure single-subject / single-theme architecture**: every pull request addresses **exactly one functional concern, one bug, or one subsystem**, ordered from uniprocessor bugfixes to full SMP multi-core integration.

```

+-----+
|          TRUE SINGLE-THEME PULL REQUEST PROGRESSION ARCHITECTURE          |
+-----+

--- PHASE 1: ATOMIC UNIPROCESSOR BUGFIXES & CONFORMANCE (ZERO SMP MACROS) ---

[PR #1: POSIX dirent.h ISO C99/C11 Empty Struct & glibc Guards]
|
v

[PR #2: Intrusive List (lists.h) Iterator Member Access Semantics]
|
v

[PR #3: Newlib Syscall Return Type Macro (c-syscalls-aliases-standard.h)]
|
v

[PR #4: Thread Suspend Symbol Export for CMSIS C Wrapper (os-thread.cpp)]
|
v

[PR #5: Memory Usable Size & Calloc Multiplication Overflow Guard]
|
v

[PR #6: ISO C++17 Aligned Memory Allocation Operators (operator new/delete)]
|
v

[PR #7: Thread-Safe Meyers Singletons for Error Categories]
|
v

[PR #8: Steady Clock 64-Bit Tick Duration Overflow Protection]
|
v

[PR #9: CMSIS-RTOS v1 C API Wrapper Cleanups (os-c-wrapper.cpp)]
|
v

[PR #10: POSIX I/O File Descriptor Table Critical Section Mutexing]
|
v

[PR #11: POSIX I/O Filesystem Mount & Network Stack List Mutexing]
|
v

[PR #12: Block Device Partition Size Validation Check]
|
v

[PR #13: ARMv8-M Architecture Semihosting Trap Instruction Guards]
|

```

```

    v
[PR #14: ARMv8-M Security Extension (TrustZone) SecureFault Handler]
    |
    v
[PR #15: Semihosting Syscall fstat() Character Device Fallback Mode]
    |
    v
[PR #16: Software Timer Callback Execution Decoupling & Drift Prevention]
    |
    v
[PR #17: POSIX Mutex Priority Ceiling & Dynamic Inheritance Boost]
    |
    v
[PR #18: Thread Lifecycle State Machine (state::destroying) & Reaper]
    |
    v
[PR #19: Condition Variable Lost-Signal Elimination & Clock Binding]
    |
    v
[PR #20: C++ std::thread Native Handle Synchronization & Functor Lifetime]
    |
    v
[PR #21: Message Queue Preemptive Rescheduling on Wakeups]
    |
    v
[PR #22: High-Resolution Monotonic Clock Port Synchronization (K + C + P)]

--- PHASE 2: MULTI-CORE SMP INFRASTRUCTURE & GENUINE HOST VALIDATION ---

[PR #23: Multi-Core SMP Kernel Infrastructure, Ports & native-smp Host]

--- PHASE 3: RELEASES, ARCHITECTURE DISSOLUTION & SILICON EXTENSIONS ---

[PR #24: Port Package Semantic Releases (posix-arch v1.1.0, cortexm v1.2.0)]
    |
    v
[PR #25: History-Preserving Devices Repository Subtree Dissolution]
    |
    v
[PR #26: Modern Multi-Core Silicon Ports (ARM Cortex-M33 & RP2350)]
    |
    v
[PR #27: Modular Add-Only CMake Build Architecture & Cross Toolchains]

--- PHASE 4: MULTI-CORE TEST SUITES, MULTI-ARCH PLATFORMS & CI INTEGRATION ---

```

```

[PR #28: Multi-Core Hardware FPU Context Switch Stress Suite]
|
v
[PR #29: Multi-Architecture 4-Core QEMU Platforms & Composite CI Action]

--- PHASE 5: DOCUMENTATION, FINAL MERGE & SILICON WIRING ---

[PR #30: Architectural Documentation & Specification Runbooks Suite]
|
v
[PR #31: Final Master Reconciliation, CI Metadata Restoration & Cleanup]
|
v
[PR #32: Dedicated Hardware Silicon Platform Wiring (Raspberry Pi Pico 2)]
+-----+

```

2. Catalog of Pure Single-Theme Pull Requests

PR #1: POSIX `dirent.h` ISO C Empty Struct Conformance & glibc Inclusion Guards

- **Single Theme:** ISO C99/C11 syntax validity and host glibc macro collision prevention.
- **Repository:** `micro-os-plus-iii`
- **File Touched:** `include/cmsis-plus/posix/dirent.h`
- **Problem & Solution:**
 - `typedef struct { ; } DIR;` violates ISO C99/C11 §6.7.2.1 (struct must have at least one named member) and triggers `-Wextra-semi` under strict compilation flags.
 - Adding `int reserved;` fixes syntax validity. Modern glibc (`__USE_MISC`/ISO C23) header guards prevent double-declaration errors on Linux hosts.
- **Exact Unified Diff Applied:**

```

--- a/include/cmsis-plus/posix/dirent.h
+++ b/include/cmsis-plus/posix/dirent.h
@@ -32,7 +32,9 @@
 #if !defined(CMSIS_PLUS_POSIX_DIRENT_H_)
 #define CMSIS_PLUS_POSIX_DIRENT_H_

+#if !defined(__USE_MISC) && !defined(_DIRENT_H)
+typedef struct
+{
+  ;
+  int reserved;
+} DIR;
+#endif

```

- **GitHub CLI:**

```

gh pr create --repo micro-os-plus/micro-os-plus-iii --base xpack-development --head dan-maio:fix/
posix-dirent-c-conformance --title "fix(posix): dirent.h ISO C99/C11 empty struct conformance &
glibc guards" --body "Adds 'int reserved;' to DIR struct to comply with ISO C99/C11 §6.7.2.1 and
adds standard glibc guards."

```

PR #2: Intrusive Double-List (`lists.h`) Iterator Member Access Semantics

- **Single Theme:** Modern C++ standard template iterator concept conformance.
- **Repository:** `micro-os-plus-iii`
- **File Touched:** `include/cmsis-plus/utils/lists.h`
- **Problem & Solution:**
 - `double_list_iterator` postfix/prefix operators were accessing `node_->next` / `node_->prev` as member variables instead of member function calls `node_->next()` / `node_->prev()`.
 - Fixes iterator method call semantics, ensuring template instantiation validity under modern C++ standards.
- **Exact Unified Diff Applied:**

```

--- a/include/cmsis-plus/utils/lists.h
+++ b/include/cmsis-plus/utils/lists.h
@@ -415,7 +415,7 @@ namespace os
    double_list_iterator
    operator++ (int)
    {
-       double_list_iterator tmp = *this;
-       node_ = node_>next;
+       double_list_iterator tmp = *this;
+       node_ = node_>next ();
        return tmp;
    }

@@ -429,7 +429,7 @@ namespace os
    double_list_iterator
    operator-- (int)
    {
-       double_list_iterator tmp = *this;
-       node_ = node_>prev;
+       double_list_iterator tmp = *this;
+       node_ = node_>prev ();
        return tmp;
    }

```

- **GitHub CLI:**

```

gh pr create --repo micro-os-plus/micro-os-plus-iii --base xpack-development --head dan-maio:fix/
list-iterator-semantics --title "fix(utils): double_list_iterator member function access
semantics" --body
"Corrects double_list_iterator operators to invoke node_>next() / node_>prev() member functions."

```

PR #3: Newlib C Syscall Weak Aliases Return Type Macro

- **Single Theme:** Newlib 64-bit libc syscall signature compatibility.
- **Repository:** `micro-os-plus-iii`
- **File Touched:** `include/cmsis-plus/posix-io/c-syscalls-aliases-standard.h`
- **Problem & Solution:**
 - Newlib on 64-bit architectures uses `_READ_WRITE_RETURN_TYPE` (`int`), whereas POSIX defines `ssize_t` (`long`).
 - Maps `CMSIS_PLUS_POSIX_IO_RW_RETURN_TYPE` to newlib's `_READ_WRITE_RETURN_TYPE`, eliminating conflicting declaration compiler errors.

- **Exact Unified Diff Applied:**

```
--- a/include/cmsis-plus/posix-io/c-syscalls-aliases-standard.h
+++ b/include/cmsis-plus/posix-io/c-syscalls-aliases-standard.h
@@ -40,7 +40,11 @@
 #pragma GCC diagnostic ignored "-Wredundant-decls"
 #endif

+#if defined(_READ_WRITE_RETURN_TYPE)
+#define CMSIS_PLUS_POSIX_IO_RW_RETURN_TYPE _READ_WRITE_RETURN_TYPE
+#else
 #define CMSIS_PLUS_POSIX_IO_RW_RETURN_TYPE ssize_t
+#endif

CMSIS_PLUS_POSIX_IO_RW_RETURN_TYPE
_read (int fildes, void* buf, size_t nbyte);
```

- **GitHub CLI:**

```
gh pr create --repo micro-os-plus/micro-os-plus-iii --base xpack-development --head dan-maio:fix/
newlib-syscall-return-type --title "fix(posix-io): harmonize syscall return types with newlib
_READ_WRITE_RETURN_TYPE"
```

PR #4: Thread Suspend External Symbol Export for CMSIS C Wrapper

- **Single Theme:** Linker external symbol visibility for thread suspension.
- **Repository:** `micro-os-plus-iii`
- **File Touched:** `src/rtos/os-thread.cpp`
- **Problem & Solution:**
 - `this_thread::suspend()` was defined with out-of-line `inline` in `.cpp`, preventing an exported external symbol from being generated.
 - Removing `inline` allows the CMSIS-RTOS C API wrapper to link `this_thread::suspend()` without undefined reference errors.
- **Exact Unified Diff Applied:**

```

--- a/src/rtos/os-thread.cpp
+++ b/src/rtos/os-thread.cpp
@@ -134,7 +134,7 @@ namespace os
     namespace this_thread
     {
-        inline void
+        void
         suspend (void)
         {
             port::scheduler::reschedule ();

```

- **GitHub CLI:**

```

gh pr create --repo micro-os-plus/micro-os-plus-iii --base xpack-development --head dan-maio:fix/
thread-suspend-symbol-export --title "fix(rtos): export out-of-line this_thread::suspend symbol
for C wrapper"

```

PR #5: Memory Allocator Usable Size & Integer Overflow Protection

- **Single Theme:** Memory resource introspection and heap integer multiplication overflow protection.
- **Repository:** `micro-os-plus-iii`
- **Files Touched:** `os-memory.*`, `first-fit-top.*`, `lifo.cpp`, `block-pool.cpp`, `malloc.cpp`
- **Problem & Solution:**
 - Implements `memory_resource::do_usable_size()` across all allocators simultaneously to eliminate undefined references.
 - Protects `calloc()` in `malloc.cpp` against integer multiplication overflow (`nelem * elbytes > SIZE_MAX`).
 - Wraps pointer arithmetic with `-Wcast-align` and `-Wunsafe-buffer-usage` pragmas.
- **Exact Unified Diff Applied:**

```

--- a/include/cmsis-plus/rtos/os-memory.h
+++ b/include/cmsis-plus/rtos/os-memory.h
@@ -140,6 +140,9 @@ namespace os
    virtual void*
        do_allocate (std::size_t bytes, std::size_t alignment) = 0;

+    virtual std::size_t
+        do_usable_size (void* addr) noexcept;
+
--- a/src/libc/stdlib/malloc.cpp
+++ b/src/libc/stdlib/malloc.cpp
@@ -72,6 +72,11 @@ extern "C"
    void*
        calloc (size_t nelem, size_t elbytes)
    {
+    if (nelem != 0 && elbytes > (SIZE_MAX / nelem))
+    {
+        errno = ENOMEM;
+        return nullptr;
+    }
        size_t size = nelem * elbytes;

```

- **GitHub CLI:**

```

gh pr create --repo micro-os-plus/micro-os-plus-iii --base xpack-development --head dan-maio:feat/
memory-usable-size-overflow-guard --title "feat(memory): implement do_usable_size() and guard
calloc against integer overflow"

```

PR #6: ISO C++17 Aligned Memory Allocation Operators

- **Single Theme:** C++17 over-aligned dynamic memory allocation overloads.
- **Repository:** `micro-os-plus-iii`
- **File Touched:** `src/libcpp/new.cpp`
- **Problem & Solution:**
- Implements `operator new(std::size_t, std::align_val_t)` and `operator delete(void*, std::align_val_t)` routing to the underlying RTOS memory manager.
- **Exact Unified Diff Applied:**

```

--- a/src/libcpp/new.cpp
+++ b/src/libcpp/new.cpp
@@ -70,4 +70,18 @@ void
operator delete[] (void* ptr, const std::nothrow_t&) noexcept
{
    os::rtos::memory::free_out_of_memory (ptr);
}
+
+#if __cplusplus >= 201703L
+void*
+operator new (std::size_t size, std::align_val_t al)
+{
+    return os::rtos::memory::alloc_out_of_memory (size, static_cast<std::size_t>(al));
+}
+
+void
+operator delete (void* ptr, std::align_val_t) noexcept
+{
+    os::rtos::memory::free_out_of_memory (ptr);
+}
+#endif

```

- **GitHub CLI:**

```

gh pr create --repo micro-os-plus/micro-os-plus-iii --base xpack-development --head dan-maio:feat/
cpp17-aligned-new-delete --title "feat(libcpp): implement ISO C++17 aligned operator new and
delete"

```

PR #7: Thread-Safe Meyers Singletons for Error Categories

- **Single Theme:** Standard library error category thread-safety and compiler exit-destructor diagnostics.
- **Repository:** `micro-os-plus-iii`
- **File Touched:** `src/libcpp/system-error.cpp`
- **Problem & Solution:**
- Implements thread-safe Meyers singletons for `generic_category()` and `system_category()` with Clang `-Wexit-time-destructors` suppression.
- **Exact Unified Diff Applied:**

```

--- a/src/libcpp/system-error.cpp
+++ b/src/libcpp/system-error.cpp
@@ -45,12 +45,18 @@ namespace os
     const error_category&
     generic_category (void) noexcept
     {
+##pragma clang diagnostic push
+##pragma clang diagnostic ignored "-Wexit-time-destructors"
         static const generic_error_category generic_category_instance;
+##pragma clang diagnostic pop
         return generic_category_instance;
     }

     const error_category&
     system_category (void) noexcept
     {
+##pragma clang diagnostic push
+##pragma clang diagnostic ignored "-Wexit-time-destructors"
         static const system_error_category system_category_instance;
+##pragma clang diagnostic pop
         return system_category_instance;
     }

```

- **GitHub CLI:**

```

gh pr create --repo micro-os-plus/micro-os-plus-iii --base xpack-development --head dan-maio:fix/
system-error-meyers-singletons --title "fix(libcpp): Meyers singletons for error categories with
exit-time destructor suppression"

```

PR #8: Steady Clock 64-Bit Duration Multiplicative Overflow Protection

- **Single Theme:** Long-uptime duration integer overflow prevention.
- **Repository:** `micro-os-plus-iii`
- **File Touched:** `src/libcpp/chrono.cpp`
- **Problem & Solution:**
 - Rewrites tick multiplication as `(cycles / freq) * 1e9 + ((cycles % freq) * 1e9) / freq`, preventing 64-bit integer overflow during multi-year uptimes.
- **Exact Unified Diff Applied:**

```

--- a/src/libcpp/chrono.cpp
+++ b/src/libcpp/chrono.cpp
@@ -74,7 +74,8 @@ namespace os
    highres_clock_traits::cycles_t cycles = highres_clock_traits::now ();
    highres_clock_traits::frequency_t freq = highres_clock_traits::frequency ();

-    return time_point (duration ((cycles * 1000000000ull) / freq));
+    return time_point (duration (
+        (cycles / freq) * 1000000000ull + ((cycles % freq) * 1000000000ull) / freq));
    }

```

- **GitHub CLI:**

```

gh pr create --repo micro-os-plus/micro-os-plus-iii --base xpack-development --head dan-maio:fix/
chrono-duration-overflow --title "fix(libcpp): prevent 64-bit integer duration overflow in steady
clock conversions"

```

PR #9: CMSIS-RTOS v1 C API Wrapper Cleanups

- **Single Theme:** CMSIS-RTOS specification conformance in C wrapper.
- **Repository:** `micro-os-plus-iii`
- **File Touched:** `src/rtos/os-c-wrapper.cpp`
- **Problem & Solution:**
 - Fixes default timer mode to one-shot (`osTimerOnce`); ensures polymorphic deletion of mutexes/semaphores; casts milliseconds to `uint64_t` prior to `* 1000u`.
- **Exact Unified Diff Applied:**

```

--- a/src/rtos/os-c-wrapper.cpp
+++ b/src/rtos/os-c-wrapper.cpp
@@ -310,7 +310,7 @@ osTimerCreate (const osTimerDef_t* timer_def, os_timer_type type, void* arg)
    if (timer_def == NULL)
    {
        return NULL;
    }
-   os_timer_type actual_type = osTimerPeriodic;
+   os_timer_type actual_type = osTimerOnce;
    if (type == osTimerPeriodic)
    {
        actual_type = osTimerPeriodic;
@@ -450,7 +450,7 @@ osStatus
    osMutexDelete (osMutexId mutex_id)
    {
-   delete (os::rtos::mutex*) mutex_id;
+   delete reinterpret_cast<os::rtos::mutex*> (mutex_id);
        return osOK;
    }

```

- **GitHub CLI:**

```

gh pr create --repo micro-os-plus/micro-os-plus-iii --base xpack-development --head dan-maio:fix/
cmsis-c-wrapper-cleanups --title "fix(rtos): CMSIS-RTOS v1 one-shot timer default, polymorphic
delete & 64-bit timeouts"

```

PR #10: POSIX I/O File Descriptor Table Critical Section Mutexing

- **Single Theme:** Thread-safe file descriptor allocation and deallocation.
- **Repository:** `micro-os-plus-iii`
- **File Touched:** `src/posix-io/file-descriptors-manager.cpp`
- **Problem & Solution:**
 - Protects file descriptor table allocation and deallocation under `interrupts::critical_section` locks, preventing descriptor corruption under concurrent `open()` / `close()`.
- **Exact Unified Diff Applied:**

```

--- a/src/posix-io/file-descriptors-manager.cpp
+++ b/src/posix-io/file-descriptors-manager.cpp
@@ -76,6 +76,7 @@ namespace os
    auto* const io = socket (family, type, protocol);
    if (io != nullptr)
    {
+       rtos::interrupts::critical_section cs;
        const int fd = allocate (io);
        if (fd < 0)
        {
@@ -130,6 +131,7 @@ namespace os
        int
        file_descriptors_manager::allocate (io* io)
        {
+       rtos::interrupts::critical_section cs;
        for (std::size_t i = 0; i < size_; ++i)
        {
            if (descriptors_array[i] == nullptr)

```

- **GitHub CLI:**

```

gh pr create --repo micro-os-plus/micro-os-plus-iii --base xpack-development --head dan-maio:fix/
posix-io-fd-mutexing --title "fix(posix-io): protect file descriptor table under critical section
locks"

```

PR #11: POSIX I/O Filesystem Mount & Network Stack List Mutexing

- **Single Theme:** Thread-safe filesystem mount and network stack linked list management.
- **Repositories:** `micro-os-plus-iii`
- **Files Touched:** `include/cmsis-plus/posix-io/file-system.h`, `include/cmsis-plus/posix-io/net-stack.h`
- **Problem & Solution:**
 - Wraps global filesystem mount lists and network stack lists with mutexes to eliminate list corruption under concurrent access.
- **Exact Unified Diff Applied:**

```

--- a/include/cmsis-plus/posix-io/file-system.h
+++ b/include/cmsis-plus/posix-io/file-system.h
@@ -82,6 +82,7 @@ namespace os
    static file_system_mount_list&
    mount_list (void);

+    static rtos::mutex&
+    mount_list_mutex (void);
--- a/include/cmsis-plus/posix-io/net-stack.h
+++ b/include/cmsis-plus/posix-io/net-stack.h
@@ -75,6 +75,7 @@ namespace os
    static net_stack_list&
    stack_list (void);

+    static rtos::mutex&
+    stack_list_mutex (void);

```

- **GitHub CLI:**

```

gh pr create --repo micro-os-plus/micro-os-plus-iii --base xpack-development --head dan-maio:fix/
posix-io-mount-net-locking --title
"fix(posix-io): mutex protection for filesystem mount lists and net-stack tables"

```

PR #12: Block Device Partition Size Validation Check

- **Single Theme:** Block device partition bounds validation.
- **Repository:** `micro-os-plus-iii`
- **File Touched:** `src/posix-io/block-device.cpp`
- **Problem & Solution:**
 - Corrects an inverted partition size check, returning `EINVAL` when registering a partition of zero blocks.
- **Exact Unified Diff Applied:**

```

--- a/src/posix-io/block-device.cpp
+++ b/src/posix-io/block-device.cpp
@@ -112,7 +112,7 @@ namespace os
    std::size_t num_blocks)
    {
-    if (num_blocks == 0)
+    if (num_blocks == 0 || (offset_blocks + num_blocks) > blocks_)
    {
        errno = EINVAL;
        return -1;

```

- **GitHub CLI:**

```

gh pr create --repo micro-os-plus/micro-os-plus-iii --base xpack-development --head dan-maio:fix/
block-device-size-check --title "fix(posix-io): correct zero-size partition check on block
devices"

```

PR #13: ARMv8-M Architecture Semihosting Trap Instruction Guards

- **Single Theme:** ARMv8-M mainline/baseline semihosting trap instructions.
- **Repository:** `micro-os-plus-iii`
- **File Touched:** `include/cmsis-plus/arm/semihosting.h`
- **Problem & Solution:**
- Adds `__ARM_ARCH_8M_MAIN__` and `__ARM_ARCH_8M_BASE__` macro guards for semihosting trap instructions.
- **Exact Unified Diff Applied:**

```

--- a/include/cmsis-plus/arm/semihosting.h
+++ b/include/cmsis-plus/arm/semihosting.h
@@ -62,7 +62,8 @@
    #if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) || \
-    defined(__ARM_ARCH_6M__)
+    defined(__ARM_ARCH_6M__) || defined(__ARM_ARCH_8M_BASE__) || \
+    defined(__ARM_ARCH_8M_MAIN__)

    #define OS_ARM_SEMIHOSTING_SYS_CALL_OPCODE "bkpt 0xAB"

```

- **GitHub CLI:**

```
gh pr create --repo micro-os-plus/micro-os-plus-iii --base xpack-development --head dan-maio:fix/armv8m-semihosting-guards --title "fix(arm): ARMv8-M semihosting trap instruction architecture guards"
```

PR #14: ARMv8-M Security Extension (TrustZone) SecureFault Handler

- **Single Theme:** ARM TrustZone security violation fault handling.
- **Repository:** `micro-os-plus-iii`
- **File Touched:** `src/startup/exception-handlers.c`
- **Problem & Solution:**
- Implements `SecureFault_Handler` exception handler for ARMv8-M Security Extension and weak `os_board_console_mirror()` hook.
- **Exact Unified Diff Applied:**

```
--- a/src/startup/exception-handlers.c
+++ b/src/startup/exception-handlers.c
@@ -95,6 +95,14 @@ HardFault_Handler (void)
 }

+#if defined(__ARM_FEATURE_CMSE) && (__ARM_FEATURE_CMSE >= 3)
+void __attribute__((section(".after_vectors"), weak, naked))
+SecureFault_Handler (void)
+{
+ while (1) { ; }
+}
+#endif
```

- **GitHub CLI:**

```
gh pr create --repo micro-os-plus/micro-os-plus-iii --base xpack-development --head dan-maio:feat/armv8m-securefault-handler --title "feat(startup): implement SecureFault_Handler and weak console mirror hook"
```

PR #15: Semihosting Syscall `fstat()` Character Device Fallback Mode

- **Single Theme:** Semihosting filesystem attribute fallback handling.
- **Repository:** `micro-os-plus-iii`
- **File Touched:** `src/semihosting/c-syscalls-semihosting.cpp`
- **Problem & Solution:**

- Assigns `S_IFCHR` (character device mode) only when file attributes have not been explicitly set.
- **Exact Unified Diff Applied:**

```
--- a/src/semihosting/c-syscalls-semihosting.cpp
+++ b/src/semihosting/c-syscalls-semihosting.cpp
@@ -145,7 +145,7 @@ namespace os
     buf->st_mode = S_IFCHR;
     buf->st_blksize = 0;
-    buf->st_mode = S_IFCHR;
+    if (buf->st_mode == 0) { buf->st_mode = S_IFCHR; }
     return 0;
```

- **GitHub CLI:**

```
gh pr create --repo micro-os-plus/micro-os-plus-iii --base xpack-development --head dan-maio:fix/
semihosting-fstat-fallback --title "fix(semihosting): assign S_IFCHR in fstat only when file type
is unconfigured"
```

PR #16: Software Timer Callback Execution Decoupling & Drift Prevention

- **Single Theme:** Deadlock-free software timer callback execution and periodic drift elimination.
- **Repository:** `micro-os-plus-iii`
- **Files Touched:** `src/rtos/os-timer.cpp`, `src/rtos/internal/os-lists.cpp`, `include/cmsis-plus/rtos/os-thread.h`
- **Problem & Solution:**
 - Invokes timer callbacks **outside the scheduler critical section**, preventing deadlock when callbacks acquire mutexes.
 - Re-arms periodic timers based on scheduled expiry rather than execution completion, eliminating drift.
- **Exact Unified Diff Applied:**

```

--- a/src/rtos/os-timer.cpp
+++ b/src/rtos/os-timer.cpp
@@ -185,9 +185,12 @@ namespace os
    timer->state_ = state::running;
+    // Unlock scheduler before calling user callback to avoid deadlocks
+    scheduler::unlock ();
+    (*(timer->func_)) (timer->func_args_);
+    scheduler::lock ();

    if (timer->type_ == type::periodic)
    {
+        // Re-arm from planned expiration to eliminate cumulative drift
        timer->schedule_next ();
    }

```

- **GitHub CLI:**

```

gh pr create --repo micro-os-plus/micro-os-plus-iii --base xpack-development --head dan-maio:fix/
timer-callback-decoupling --title "fix(rtos): execute software timer callbacks outside critical
section & eliminate drift"

```

PR #17: POSIX Mutex Priority Ceiling & Dynamic Boost Protocol

- **Single Theme:** Real-time priority ceiling protocol and dynamic priority inheritance tracking.
- **Repository:** `micro-os-plus-iii`
- **File Touched:** `src/rtos/os-mutex.cpp`
- **Problem & Solution:**
 - Elevates caller priority to ceiling **before** granting mutex ownership, closing the preemption inversion window.
 - Dynamically tracks maximum boost among multiple waiters and restores base priority on unlock.
- **Exact Unified Diff Applied:**

```

--- a/src/rtos/os-mutex.cpp
+++ b/src/rtos/os-mutex.cpp
@@ -245,6 +245,11 @@ namespace os
    if (protocol_ == protocol::protect)
    {
+        // Raise priority to ceiling BEFORE acquiring mutex
+        if (current_thread->priority () < priority_ceiling_)
+        {
+            current_thread->priority (priority_ceiling_);
+        }
    }

```

- **GitHub CLI:**

```

gh pr create --repo micro-os-plus/micro-os-plus-iii --base xpack-development --head dan-maio:feat/
mutex-priority-ceiling-protocol --title "feat(rtos): POSIX priority ceiling protocol & dynamic
priority boost tracking"

```

PR #18: Thread Lifecycle State Machine (`state::destroying`) & Idle Reaper

- **Single Theme:** Safe asynchronous thread resource reclamation and atomic join synchronization.
- **Repository:** `micro-os-plus-iii`
- **Files Touched:** `os-thread.*`, `os-idle.cpp`, `os-c-decls.h`
- **Problem & Solution:**
 - Introduces `state::destroying = 7` to eliminate use-after-free races during thread cleanup.
 - `thread::join()` blocks atomically until the thread is unlinked from the scheduler.
 - Idle thread reaper safely frees dynamic stacks after execution termination.
- **Exact Unified Diff Applied:**

```

--- a/include/cmsis-plus/rtos/os-thread.h
+++ b/include/cmsis-plus/rtos/os-thread.h
@@ -88,6 +88,7 @@ namespace os
    ready = 3,
    running = 4,
    suspended = 5,
-   terminated = 6
+   terminated = 6,
+   destroying = 7
};

--- a/src/rtos/os-idle.cpp
+++ b/src/rtos/os-idle.cpp
@@ -42,6 +42,7 @@ namespace os
    while (true)
    {
+       // Reclaim thread stacks and control blocks marked in destroying state
+       rtos::scheduler::reap_destroyed_threads ();
        port::waiting::wait_for_interrupt ();
    }

```

- **GitHub CLI:**

```

gh pr create --repo micro-os-plus/micro-os-plus-iii --base xpack-development --head dan-maio:feat/
thread-lifecycle-destroying-reaper --title "feat(rtos): thread state::destroying (7), atomic join
& idle thread stack reaper"

```

PR #19: Condition Variable Lost-Signal Elimination & Clock Binding

- **Single Theme:** Lost-signal race elimination in condition variables and clock attribute binding.
- **Repository:** `micro-os-plus-iii`
- **Files Touched:** `os-condvar.*`, `instrumentation.h`, `os-c-decls.h`
- **Problem & Solution:**
 - Atomically enqueues thread to wait list under scheduler lock before releasing user mutex, eliminating lost wakeups.
 - Binds `timed_wait()` to configurable attribute clocks (`os::rtos::clock*`).
- **Exact Unified Diff Applied:**

```

--- a/src/rtos/os-condvar.cpp
+++ b/src/rtos/os-condvar.cpp
@@ -155,7 +155,9 @@ namespace os
    {
+       scheduler::lock ();
        list_.link (*current_thread);
        mutex->unlock ();
-       port::scheduler::reschedule ();
+       current_thread->suspend_locked ();
+       scheduler::unlock ();
        mutex->lock ();

```

- **GitHub CLI:**

```

gh pr create --repo micro-os-plus/micro-os-plus-iii --base xpack-development --head dan-maio:fix/
condvar-atomic-wait-clock-binding --title "fix(rtos): eliminate condvar lost-signal race & bind
timed_wait to attribute clock"

```

PR #20: C++ `std::thread` Native Handle Synchronization & Functor Lifetime

- **Single Theme:** ISO C++ standard thread wrapper join semantics and closure memory safety.
- **Repository:** `micro-os-plus-iii`
- **Files Touched:** `src/libcpp/thread-cpp.h`, `include/cmsis-plus/estd/thread_internal.h`
- **Problem & Solution:**
- `std::thread::join()` waits directly on the native RTOS thread handle.
- Retains functor object lifetime in wrapper structure until completion, preventing lambda capture corruption.
- **Exact Unified Diff Applied:**

```

--- a/src/libcpp/thread-cpp.h
+++ b/src/libcpp/thread-cpp.h
@@ -92,7 +92,8 @@ namespace std
    void
    thread::join ()
    {
-       // Wait on native handle
+       if (id_ != id ())
+       {
+           native_handle ()->join ();
+           id_ = id ();
+       }
    }

```

- **GitHub CLI:**

```

gh pr create --repo micro-os-plus/micro-os-plus-iii --base xpack-development --head dan-maio:fix/
cpp-std-thread-functor-lifetime --title "fix(libcpp): std::thread native handle join barrier &
functor lifetime retention"

```

PR #21: Message Queue Preemptive Rescheduling on Wakeups

- **Single Theme:** Immediate scheduler preemption upon message queue wakeups.
- **Repository:** `micro-os-plus-iii`
- **File Touched:** `src/rtos/os-mqueue.cpp`
- **Problem & Solution:**
- Calls `port::scheduler::reschedule()` immediately after message send/receive wakes a higher-priority thread across all 6 code paths.
- **Exact Unified Diff Applied:**

```

--- a/src/rtos/os-mqueue.cpp
+++ b/src/rtos/os-mqueue.cpp
@@ -215,6 +215,7 @@ namespace os
    if (thread_woken)
    {
+       port::scheduler::reschedule ();
    }
    return result::ok;

```

- **GitHub CLI:**

```
gh pr create --repo micro-os-plus/micro-os-plus-iii --base xpack-development --head dan-maio:fix/
mqueue-preemptive-reschedule --title "fix(rtos): invoke port::scheduler::reschedule() after
message queue wakeups"
```

PR #22: High-Resolution Monotonic Clock Port Synchronization

- **Single Theme:** Cross-repository monotonic hardware clock interface and SysTick overflow detection.
- **Repositories:** `micro-os-plus-iii`, `micro-os-plus-iii-cortexm`, `micro-os-plus-iii-posix-arch`
- **Files Touched:** Kernel `os-decls.h` & `os-clocks.cpp`, Cortex-M `os-inlines.h`, POSIX-Arch `os-inlines.h`
- **Problem & Solution:**
- Kernel declares and calls `has_hardware_counter()`.
- Cortex-M fixes SysTick pending overflow inspection (`SCB->ICSR & SCB_ICSR_PENDSTSET_Msk`).
- POSIX-arch implements host `CLOCK_MONOTONIC` timestamping using `timespec tp`; (`-Werror=redundant-tags`).
- **Exact Unified Diff Applied:**

```
--- a/micro-os-plus-iii-cortexm/include/cmsis-plus/rtos/port/os-inlines.h
+++ b/micro-os-plus-iii-cortexm/include/cmsis-plus/rtos/port/os-inlines.h
@@ -110,7 +110,8 @@ namespace os
     static inline bool
     has_hardware_counter (void)
     {
-        return true;
+        // Inspect SysTick pending overflow interrupt
+        return (SCB->ICSR & SCB_ICSR_PENDSTSET_Msk) != 0;
     }
--- a/micro-os-plus-iii-posix-arch/include/cmsis-plus/rtos/port/os-inlines.h
+++ b/micro-os-plus-iii-posix-arch/include/cmsis-plus/rtos/port/os-inlines.h
@@ -85,7 +85,7 @@ namespace os
     static inline uint64_t
     hardware_counter (void)
     {
-        struct timespec tp;
+        timespec tp; // -Werror=redundant-tags clean
+        clock_gettime (CLOCK_MONOTONIC, &tp);
+        return static_cast<uint64_t> (tp.tv_sec) * 1000000000ull + tp.tv_nsec;
     }
```

- **GitHub CLI:**

```
gh pr create --repo micro-os-plus/micro-os-plus-iii --base xpack-development --head dan-maio:feat/
highres-clock-port-sync --title "feat(rtos): cross-repo high-resolution monotonic clock port
synchronization"
```

PR #23: Multi-Core SMP Kernel Infrastructure, Port Contracts & `native-smp`

- **Single Theme:** Multi-core SMP scheduling engine, recursive kernel spinlock, 5-stage stack handshake, and host dual-core platform.
- **Repositories:** `micro-os-plus-iii`, `micro-os-plus-iii-cortexm`, `micro-os-plus-iii-posix-arch`
- **Base Branch:** `step/13` | **Head Branch:** `step/14`
- **Problem & Solution:**
 - Interwoven SMP subsystems (scheduler, threads, idle, locks, port contracts) must land together for compilation consistency.
 - Implements 5-stage stack pointer claim/publish protocol preventing stack corruption across cores.
 - Adds `tests/platforms/native-smp/` setting `OS_USE_SMP_SCHEDULER=1` via `target_compile_definitions` for real host dual-core verification.
- **Exact Unified Diff Applied:**

```

--- a/include/cmsis-plus/rtos/os-sched.h
+++ b/include/cmsis-plus/rtos/os-sched.h
@@ -82,6 +82,10 @@ namespace os
    #if defined(OS_USE_SMP_SCHEDULER)
+       static thread* volatile current_thread_[OS_NCPU];
+       static ready_list ready_list_[OS_NCPU];
+       static port::spinlock_t _smp_klock;
    #else
        static thread* volatile current_thread_;
--- a/src/rtos/os-core.cpp
+++ b/src/rtos/os-core.cpp
@@ -140,6 +140,15 @@ namespace os
    #if defined(OS_USE_SMP_SCHEDULER)
+   void
+   scheduler::lock (void)
+   {
+       port::spinlock_acquire (&_smp_klock);
+   }
+   void
+   scheduler::unlock (void)
+   {
+       port::spinlock_release (&_smp_klock);
+   }
    #endif

```

- **GitHub CLI:**

```

gh pr create --repo micro-os-plus/micro-os-plus-iii --base step/13 --head dan-maio:step/14 --
title "feat(smp): multi-core SMP scheduler, port contracts & native-smp dual-core platform"

```

PR #24: Port Package Semantic Version Releases

- **Single Theme:** Formalizing and tagging SMP port contract releases.
- **Repositories:** `micro-os-plus-iii-posix-arch`, `micro-os-plus-iii-cortexm`
- **Files Touched:** `posix-arch/package.json` (`v1.1.0`), `cortexm/package.json` (`v1.2.0`)
- **Problem & Solution:**
- Releases `posix-arch v1.1.0` and `cortexm v1.2.0` in `package.json`.
- Python release script (`release-port.sh`) bypasses `npm version` unauthorized remote push hooks.
- **Exact Unified Diff Applied:**

```

--- a/micro-os-plus-iii-posix-arch/package.json
+++ b/micro-os-plus-iii-posix-arch/package.json
@@ -3,3 +3,3 @@
- "version": "1.0.1",
+ "version": "1.1.0",
--- a/micro-os-plus-iii-cortexm/package.json
+++ b/micro-os-plus-iii-cortexm/package.json
@@ -3,3 +3,3 @@
- "version": "1.1.0",
+ "version": "1.2.0",

```

- **GitHub CLI:**

```

gh pr create --repo micro-os-plus/micro-os-plus-iii-posix-arch --base step/14 --head dan-maio:step/24 --title "release: v1.1.0"
gh pr create --repo micro-os-plus/micro-os-plus-iii-cortexm --base step/14 --head dan-maio:step/24 --title "release: v1.2.0"

```

PR #25: History-Preserving Devices Repository Subtree Dissolution

- **Single Theme:** Folding device drivers and SoC silicon files into architecture repositories.
- **Repositories:** `micro-os-plus-iii-posix-arch`, `micro-os-plus-iii-cortexm`, `micro-os-plus-iii-aarch32`, `micro-os-plus-iii-aarch64`
- **Problem & Solution:**
- Dissolves `micro-os-plus-iii-devices` using `git subtree add`, preserving 100% of commit history in owning architecture repositories.
- Injects `micro-os-plus::devices` CMake ALIAS shims for backwards compatibility.
- **Exact Unified Diff Applied:**

```

--- a/micro-os-plus-iii-cortexm/CMakeLists.txt
+++ b/micro-os-plus-iii-cortexm/CMakeLists.txt
@@ -45,6 +45,9 @@
+# Compatibility alias for dissolved devices repository
+if(NOT TARGET micro-os-plus::devices)
+  add_library(micro-os-plus::devices ALIAS micro-os-plus::cortexm)
+endif()

```

- **GitHub CLI:**

```
gh pr create --repo micro-os-plus/micro-os-plus-iii-cortexm --base xpack-development --head dan-
maio:part0-devices --title
"refactor(devices): subtree migration of STM32F4 and RP2350 SoC support from devices repo"
```

PR #26: Modern Multi-Core Silicon Ports (Cortex-M33 & RP2350)

- **Single Theme:** Hardware port implementations for ARM Cortex-M33 and Raspberry Pi RP2350.
- **Repositories:** `micro-os-plus-iii-cortexm`, `micro-os-plus-iii`
- **Files Touched:** `include-m33/`, `src/rtos/os-core-m33.cpp`, `include-rp2350/`, `src/rtos/os-core-rp2350.cpp`, `thread::resume()` IPI
- **Problem & Solution:**
 - Adds Cortex-M33 (ARMv8-M Mainline) port with Message Handling Unit (MHU) IPI signaling.
 - Adds RP2350 port with recursive `_smp_klock` backed by SIO Hardware Spinlock 0 (`0xD00000100`) and FIFO IRQ 25.
- **Exact Unified Diff Applied:**

```
--- a/micro-os-plus-iii-cortexm/src/rtos/os-core-rp2350.cpp
+++ b/micro-os-plus-iii-cortexm/src/rtos/os-core-rp2350.cpp
@@ -0,0 +1,48 @@
+#include <cmsis-plus/rtos/os.h>
+#if defined(OS_USE_SMP_SCHEDULER) && defined(TARGET_RP2350)
+namespace os::rtos::port {
+ void spinlock_acquire(spinlock_t* lock) {
+   while ((SIO_SPINLOCK0 & 1) == 0) { __wfe(); }
+ }
+ void spinlock_release(spinlock_t* lock) {
+   SIO_SPINLOCK0 = 1;
+   __sev();
+ }
+}
+#endif
```

- **GitHub CLI:**

```
gh pr create --repo micro-os-plus/micro-os-plus-iii-cortexm --base step/24 --head dan-maio:step/
25 --title "feat(cortexm): ARM Cortex-M33 and Raspberry Pi RP2350 multi-core silicon support"
```

PR #27: Modular Add-Only CMake Build Architecture & Cross Toolchains

- **Single Theme:** Modular CMake targets, cross toolchains, and `2xcortex-m33` platform.

- **Repositories:** `micro-os-plus-iii`, `micro-os-plus-iii-cortexm`
- **Files Touched:** `cmake/toolchains/`, `cmake/uos-app.cmake`, `micro-os-plus::cortexm-qemu-m33`, `tests/platforms/2xcortex-m33/`
- **Problem & Solution:**
- Adds `cmake/uos-app.cmake` and cross toolchains (`arm-none-eabi.cmake`, `aarch64-none-elf.cmake`).
- Appends `micro-os-plus::cortexm-qemu-m33` linking fat `micro-os-plus::iii`.
- Adds `tests/platforms/2xcortex-m33/` (`mps2-an521 --cpu cortex-m33 --smp 2`).
- **Exact Unified Diff Applied:**

```
--- a/CMakeLists.txt
+++ b/CMakeLists.txt
@@ -85,6 +85,9 @@
+add_subdirectory(cmake/toolchains)
+include(cmake/uos-app.cmake)
+
+## Modular targets
+add_library(micro-os-plus-iii-cortexm-qemu-m33 INTERFACE)
```

- **GitHub CLI:**

```
gh pr create --repo micro-os-plus/micro-os-plus-iii --base step/25 --head dan-maio:step/26 --
title "feat(cmake): modular add-only CMake toolchains & 2xcortex-m33 QEMU platform"
```

PR #28: Multi-Core FPU Stress Testing & Scaffolding Relocation

- **Single Theme:** Hardware FPU context switch stress testing and test runner standardization.
- **Repository:** `micro-os-plus-iii`
- **Files Touched:** `tests/sources/fp-switch/`, `tests/smp-support/`
- **Problem & Solution:**
- Adds `tests/sources/fp-switch/` (6 threads, 3000 ticks) verifying floating-point register preservation across cores.
- Absorbs root `test_smpl/` into `tests/smp-support/` (`hw_result.hpp`, `board-contract.cpp`, `run-qemu.sh`, `run-host.sh`, `run-hw.sh`).
- **Exact Unified Diff Applied:**

```

--- a/tests/cmake/global-definitions.cmake
+++ b/tests/cmake/global-definitions.cmake
@@ -45,6 +45,9 @@
+## Enable Hardware FPU Context Switch Stress Test
+option(ENABLE_FP_SWITCH_TEST "Enable 6-thread FPU stress test" OFF)
+if(ENABLE_FP_SWITCH_TEST)
+  add_compile_definitions(OS_TEST_FP_SWITCH=1)
+endif()

```

- **GitHub CLI:**

```

gh pr create --repo micro-os-plus/micro-os-plus-iii --base step/26 --head dan-maio:step/27 --
title "test(smp): hardware FPU context switch stress test & tests/smp-support absorption"

```

PR #29: Multi-Architecture 4-Core Platforms & Composite CI Action

- **Single Theme:** AArch32/AArch64 4-core QEMU platforms and `test-smp-all` action.
- **Repositories:** `micro-os-plus-iii`, `micro-os-plus-iii-aarch32`, `micro-os-plus-iii-aarch64`
- **Files Touched:** `aarch32-rpi3b`, `aarch64-rpi3b`, `test-smp-all` in `tests/package.json`
- **Problem & Solution:**
- Injects granular kernel sub-targets (`::iii-posix-io`, `::iii-semihosting`, `::iii-newlib-reent`, `::test-support`).
- Adds 4-core QEMU `raspi3b` platforms for AArch32 and AArch64.
- Registers `test-smp-all` composite action in `tests/package.json`.
- **Exact Unified Diff Applied:**

```

--- a/tests/package.json
+++ b/tests/package.json
@@ -88,6 +88,11 @@
+  "test-smp-all": {
+    "description": "Run all SMP test suites across Native, Cortex-M33, and AArch32/64",
+    "actions": [
+      "xpm run test-native-smp",
+      "xpm run test-2xcortex-m33-cmake",
+      "xpm run test-aarch32-rpi3b",
+      "xpm run test-aarch64-rpi3b"
+    ]
+  }

```

- **GitHub CLI:**

```
gh pr create --repo micro-os-plus/micro-os-plus-iii --base step/27 --head dan-maio:step/28 --
title "feat(aarch): 4-core AArch32/AArch64 platforms & test-smp-all CI composite action"
```

PR #30: Comprehensive Architectural Documentation & Specification Runbooks

- **Single Theme:** Architectural specifications, diagrams, and PDF documentation suite.
- **Repository:** `micro-os-plus-iii`
- **Files Touched:** `docs/*.md`, `docs/*.pdf`
- **Problem & Solution:**
- Synchronizes and builds complete PDF documentation suite (`MICRO-OS-PLUS-SMP-VS-SINGLECORE-ANALYSIS.pdf`, `SMP-UPSTREAM-INTEGRATION-PLAN.pdf`, `Implementation-SMP-Integration.pdf`, `pool-request.pdf`, `files-modif-by-step.pdf`, `GITHUB-PROGRESSIVE-PR-GUIDE.pdf`).
- **Exact Unified Diff Applied:**

```
--- a/docs/render-pdfs.sh
+++ b/docs/render-pdfs.sh
@@ -24,6 +24,7 @@ render_doc "SMP-UPSTREAM-INTEGRATION-PLAN.md"
   render_doc "Implementation-SMP-Integration.md"
   render_doc "pool-request.md"
   render_doc "files-modif-by-step.md"
+render_doc "GITHUB-PROGRESSIVE-PR-GUIDE.md"
```

- **GitHub CLI:**

```
gh pr create --repo micro-os-plus/micro-os-plus-iii --base step/28 --head dan-maio:step/29 --
title "docs(all): comprehensive architectural analysis, integration runbooks & PDF suite"
```

PR #31: Final Upstream Master Reconciliation & Repository Cleansing

- **Single Theme:** Upstream master branch reconciliation, pristine CI configuration restoration, and tooling cleanup.
- **Repositories:** `micro-os-plus-iii`, all ports
- **Base Branch:** `step/29` | **Head Branch:** `step/30`
- **Problem & Solution:**
- Merges `origin/xpack-development` cleanly.
- Restores byte-identical `.github/workflows/ci.yml`, `README.md`, `LICENSE`, and Doxygen templates.
- Removes transient `scripts/smp/` directory.

- **Exact Unified Diff Applied:**

```
--- a/.github/workflows/ci.yml
+++ b/.github/workflows/ci.yml
@@ -1,5 +1,5 @@
 # Clean upstream CI workflow restored byte-identical to xpack-development baseline
 name: CI
--- a/scripts/smp/ (DELETED)
- Transitory staging and unifdef automation scripts removed before final release
```

- **GitHub CLI:**

```
gh pr create --repo micro-os-plus/micro-os-plus-iii --base xpack-development --head dan-maio:step/
30 --title "chore(release): upstream master reconciliation, pristine CI restoration & cleanup"
```

PR #32: Dedicated Physical Silicon Board Wiring (Raspberry Pi Pico 2)

- **Single Theme:** Hardware platform definition and OpenOCD SWD flashing runners for physical RP2350 silicon.
- **Repositories:** `micro-os-plus-iii`, `micro-os-plus-iii-cortexm`
- **Files Touched:** `tests/platforms/cortexm-pico2/`, `os-core-rp2350.cpp`
- **Problem & Solution:**
 - Adds standalone hardware platform `tests/platforms/cortexm-pico2/` and OpenOCD SWD flashing runners.
 - Implements Core 1 secondary boot handshake sequence via SIO mailbox registers (`sio_hw->fifo_wr` / `sio_hw->fifo_rd`).
- **Exact Unified Diff Applied:**

```
--- a/tests/platforms/cortexm-pico2/CMakeLists.txt
+++ b/tests/platforms/cortexm-pico2/CMakeLists.txt
@@ -0,0 +1,32 @@
+add_executable(pico2-smp-test
+  main.cpp
+  ${CMAKE_CURRENT_LIST_DIR}/board.cpp
+)
+target_link_libraries(pico2-smp-test PRIVATE micro-os-plus::cortexm-rp2350)
```

- **GitHub CLI:**

```
gh pr create --repo micro-os-plus/micro-os-plus-iii --base step/30 --head dan-maio:step/31 --
title "feat(board): dedicated Raspberry Pi Pico 2 (RP2350) hardware platform & runner"
```

3. Self-Contained Compilation & Testing Verification Matrix

To ensure that **every single pull request compiles, links, and can be independently tested on GitHub**, every PR satisfies strict self-contained boundary rules:

4.1 The Three Dependency Classes

1. **Pure Single-File / Intra-Repo PRs:** (PRs #1–#12, #18, #19, #21, #22)
2. Self-contained in a single repository.
3. Requires no simultaneous port changes.
4. Tested directly by running `xpm run test-all` (72/72 uniprocessor test matrix).
5. **Atomic Cross-Repository Declarations & Definitions:** (PR #13, PR #14, PR #20)
6. *Rule:* A header declaration in the Kernel and its hardware implementation in the Ports must travel in the same PR stage to avoid `undefined reference` linker errors.
7. *Example (PR #13):* Kernel `os-decls.h` declares `port::clock_highres::has_hardware_counter()`, and Cortex-M & POSIX `os-inlines.h` define it. Both are committed together across repos.
8. *Example (PR #14):* Kernel SMP scheduler calls `port_cpu_id()`, and POSIX/Cortex-M ports define it. Both are committed together in `step/14`.
9. **Additive Platform & Multi-Core Suites:** (PR #17, #18, #19, #20, #23)
10. Self-contained add-only additions.
11. Tested by running `xpm run test-smp-all` alongside the baseline `xpm run test-all`.

4.2 Comprehensive Verification Matrix Per Single-Theme PR

PR #	Theme / Subject	Repos	Self-Contained?	Local Build & Test Command	Acceptance Gate
#1	POSIX <code>dirent.h</code> Conformance	Kernel	Yes (Standalone)	<code>xpm run test-all</code>	72/72 tests pass; <code>-Wextra-semi</code> clean
#2	List Iterator Member Semantics	Kernel	Yes (Standalone)	<code>xpm run test-all</code>	72/72 tests pass; intrusive list suite green
#3	Newlib Syscall Return Types	Kernel	Yes (Standalone)	<code>xpm run test-all</code>	72/72 tests pass; newlib reentrant tests pass
#4	Thread Suspend Symbol Export	Kernel	Yes (Standalone)	<code>xpm run test-all</code>	72/72 tests pass; C wrapper resolves symbol
#5	Memory Allocator Usable Size	Kernel	Yes (Cohesive 7 files)	<code>xpm run test-all</code>	72/72 tests pass; <code>malloc_usable_</code> <code>size</code> verified
#6	C++17 Aligned Allocation	Kernel	Yes (Standalone)	<code>xpm run test-all</code>	72/72 tests pass; <code>operator new</code> overloads green
#7	Meyers Error Singletons	Kernel	Yes (Standalone)	<code>xpm run test-all</code>	72/72 tests pass; <code>-Wexit-time-</code> <code>destructors</code> clean
#8	Steady Clock Duration Overflow	Kernel	Yes (Standalone)	<code>xpm run test-all</code>	72/72 tests pass; chrono math overflow-free
#9	CMSIS-RTOS C Wrapper	Kernel	Yes (Standalone)	<code>xpm run test-all</code>	72/72 tests pass; <code>cmsis-os-</code> <code>validator</code> (60/60)
#10	POSIX I/O FD Table Mutex	Kernel	Yes (Standalone)	<code>xpm run test-all</code>	72/72 tests pass; concurrent I/O stress green
#11	POSIX I/O Mount List Locking	Kernel	Yes (Standalone)	<code>xpm run test-all</code>	72/72 tests pass; filesystem mount tests green
#12	Block Device Partition Bounds	Kernel	Yes (Standalone)	<code>xpm run test-all</code>	72/72 tests pass; invalid partition returns <code>EINVAL</code>
#13	ARMv8-M Semihosting Traps	Kernel	Yes (Standalone)	<code>xpm run test-all</code>	72/72 tests pass; ARM semihosting tests pass
#14	SecureFault Exception Handler	Kernel	Yes (Standalone)	<code>xpm run test-all</code>	

PR #	Theme / Subject	Repos	Self-Contained?	Local Build & Test Command	Acceptance Gate
					72/72 tests pass; exception vectors compile
#15	Semihosting <code>fstat()</code> Mode	Kernel	Yes (Standalone)	<code>xpm run test-all</code>	72/72 tests pass; file mode attributes clean
#16	Software Timer Decoupling	Kernel	Yes (Cohesive 3 files)	<code>xpm run test-all</code>	72/72 tests pass; timer callback mutexes clean
#17	Mutex Priority Ceiling	Kernel	Yes (Standalone)	<code>xpm run test-all</code>	72/72 tests pass; <code>mutex-stress-test</code> passes
#18	Thread Lifecycle & Reaper	Kernel	Yes (Cohesive 4 files)	<code>xpm run test-all</code>	72/72 tests pass; dynamic thread reaper green
#19	CondVar Lost-Signal Elimination	Kernel	Yes (Cohesive 4 files)	<code>xpm run test-all</code>	72/72 tests pass; condvar concurrency stress green
#20	<code>std::thread</code> Handle Sync	Kernel	Yes (Cohesive 2 files)	<code>xpm run test-all</code>	72/72 tests pass; functor closure lifetime verified
#21	MQueue Preemptive Reschedule	Kernel	Yes (Standalone)	<code>xpm run test-all</code>	72/72 tests pass; message queue preemption green
#22	High-Res Clock Port Sync	K + C + P	Yes (Atomic across repos)	<code>xpm run test-all</code>	72/72 tests pass; ICSR pending fix verified
#23	Multi-Core SMP Core & Native	K + C + P	Yes (Atomic across repos)	<code>scripts/smp/verify-step.sh 14</code>	<code>unifdef</code> zero-diff + 72/72 + <code>native-smp</code> (3/3)
#24	Port Semantic Releases	P + C	Yes (Version bump only)	<code>git tag -l</code>	<code>v1.1.0</code> and <code>v1.2.0</code> local tags generated
#25	Devices Subtree Dissolution	P + C + Arch	Yes (Subtree add)	<code>xpm run build ...</code>	Subtree merged; CMake ALIAS targets resolve
#26	Modern Silicon Ports	C + K	Yes (Add-only)	<code>scripts/smp/verify-step.sh 25</code>	M33/RP2350 port targets compile cleanly
#27	Modular Add-Only CMake	K + C	Yes (Add-only)		

PR #	Theme / Subject	Repos	Self-Contained?	Local Build & Test Command	Acceptance Gate
				<code>scripts/smp/verify-step.sh</code> 26	72/72 baseline + <code>2xcortex-m33</code> (4/4) PASS
#28	FPU Context Switch Stress	Kernel	Yes (Add-only)	<code>scripts/smp/verify-step.sh</code> 27	<code>fp-switch</code> passes on <code>2xcortex-m33</code> (4/4)
#29	Multi-Arch 4-Core Platforms	K + A32 + A64	Yes (Add-only)	<code>scripts/smp/verify-step.sh</code> 28	<code>test-smp-all</code> passes (Native, M33, AArch)
#30	Documentation Suite	Kernel	Yes (Standalone)	<code>docs/render-pdfs.sh</code>	All PDFs compile cleanly without warnings
#31	Final Merge & Cleansing	All Repos	Yes (Clean merge)	<code>bash scripts/smp/finalize.sh</code>	Upstream CI metadata 100% byte-identical
#32	Pico 2 Hardware Platform	K + C	Yes (Add-only)	<code>xpm run build --config cortexm-pico2</code>	Pico 2 platform compiles for hardware SWD

4. Command-Line Rebase & Stacked PR Maintenance Cheatsheet

When working with single-theme stacked PRs on GitHub:

```
# When upstream merges PR #1 (fix/posix-dirent-c-conformance)
cd "$WORK/micro-os-plus-iii"
git fetch upstream

# Rebase PR #2 (fix/list-iterator-semantics) onto upstream
git checkout fix/list-iterator-semantics
git rebase upstream/xbpack-development

# Force push with lease
git push origin fix/list-iterator-semantics --force-with-lease

# Retarget PR #2 base on GitHub
gh pr edit <PR_2_NUMBER> --base xpack-development
```

5. Web Interface Guide: Generating and Managing PRs on GitHub.com

While the GitHub CLI (`gh pr create`) enables scriptable command-line PR generation, the **GitHub.com Web Interface** offers full visual inspection of branch comparisons, diff highlights, review comments, and stacked base retargeting.

5.1 Step-by-Step: Creating a Pull Request on GitHub.com

Step 1: Push Your Local Branch to Your Fork

Before opening a pull request on the web interface, ensure your local step branch is pushed to your remote repository on GitHub:

```
git push -u origin <branch-name>
# Examples:
# git push -u origin fix/posix-dirent-c-conformance
# git push -u origin step/01
# git push -u origin step/28
```

Step 2: Open the Upstream Repository in Your Browser

Navigate to the target upstream repository on GitHub: - Target Upstream: <https://github.com/micro-os-plus/micro-os-plus-iii> - Personal Fork: <https://github.com/dan-maio/micro-os-plus-iii>

Step 3: Click "New Pull Request" & Configure Branch Comparison

1. Navigate to the **"Pull requests"** tab at the top of the repository page.
2. Click the green **"New pull request"** button on the right.
3. If comparing across forks (e.g., from `dan-maio` to `micro-os-plus`), click the link **"compare across forks"**.
4. Configure the 4-part branch comparison bar:

```

+-----+
|          GITHUB PULL REQUEST COMPARISON BAR          |
+-----+
|
|  BASE (Target Upstream):          HEAD (Source / Fork):  |
|  -----          -----          |
|  repo:  micro-os-plus-iii      <-- repo:  dan-maio/micro-os-plus-iii |
|  branch: xpack-development      branch: fix/posix-dirent  |
|
|  -----          |
|  DROPDOWN CONTROLS VIEW:          |
|
|  base repository: [ micro-os-plus/micro-os-plus-iii v ]   |
|  base branch:    [ xpack-development          v ]         |
|
|                      ^                                     |
|                      | (merges incoming changes into)      |
|  head repository: [ dan-maio/micro-os-plus-iii      v ]   |
|  compare branch:  [ fix/posix-dirent-c-conformance  v ]   |
|
+-----+

```

- **Base repository:** `micro-os-plus/micro-os-plus-iii` (or your staging fork `dan-maio/micro-os-plus-iii`)
- **Base branch:** The baseline destination:
- For uniprocessor fixes: `xpack-development`
- For progressive stacked PRs: The preceding verified step branch (e.g., `step/27` for Step 28, or `step/28` for Step 29/30)
- **Head repository:** `dan-maio/micro-os-plus-iii`
- **Compare branch:** Your topic or step branch (e.g., `step/28`)

Step 4: Inspect the Pre-Flight Diff on the Web UI

Before clicking "Create", review the summary bar below the comparison selectors:

- **Commits count:** Should be 1 commit (for single-subject PRs).
- **Files changed:** Should list only the files intended for that specific theme.
- **Diff viewer:** Verify the green (+) additions and red (-) deletions match your expectations.

```

+-----+
| Showing 1 changed file (+2 / -1) [ Split | Unified ] |
+-----+
| diff --git a/include/cmsis-plus/posix/dirent.h ... |
| @@ -32,7 +32,9 @@ |
| #if !defined(CMSIS_PLUS_POSIX_DIRENT_H_) |
| #define CMSIS_PLUS_POSIX_DIRENT_H_ |
| |
| +#if !defined(__USE_MISC) && !defined(_DIRENT_H) |
| typedef struct |
| { |
| - ; |
| + int reserved; |
| } DIR; |
| +#endif |
+-----+

```

Step 5: Fill in Title & Description

1. Click the green **"Create pull request"** button.
2. **Title:** Enter a concise conventional commit title, e.g.: `fix(posix): dirent.h ISO C99/C11 empty struct conformance & glibc guards`
3. **Description:** Paste the problem rationale, solution summary, and verification results.
4. Ensure the checkbox **"Allow edits by maintainers"** is checked (allows upstream maintainers to push fast-forward fixes directly).
5. Click **"Create pull request"** (or use the dropdown arrow to select **"Create draft pull request"** if work is still in progress).

5.2 Managing Stacked PRs on GitHub.com (Retargeting the Base Branch)

When submitting a sequence of dependent PRs (such as `step/01` → `step/02` → ... → `step/30`), each successive PR is opened targeting the previous step branch as its base.

Retargeting When the Previous PR is Merged

When upstream merges the preceding PR (e.g., `step/01` merged into `xpack-development`):

```

+-----+
|                RETARGETING BASE BRANCH ON GITHUB.COM                |
+-----+
|                                                                    |
| 1. Open PR #2 on GitHub.com (e.g. step/02).                        |
| 2. Under the PR title, find the branch header:                      |
|                                                                    |
|    dan-maio wants to merge into [ step/01 v ] from [ step/02 ]    [ Edit ] |
|                                                                    |
| 3. Click [ Edit ] on the far right.                                v  |
| 4. Change base dropdown from [ step/01 v ] to [ xpack-development v ]. |
| 5. Click [ Save ].                                                |
|                                                                    |
| RESULT: GitHub recalculates the diff in real time against upstream! |
|                                                                    |
+-----+

```

5.3 Conducting Code Reviews & Inline Suggestions on GitHub.com

Reviewers and authors can collaborate directly within the GitHub.com web interface:

1. Files Changed Tab (`/files`)

- Toggle between **"Unified"** (inline) and **"Split"** (side-by-side) diff views using the gear icon.
- Click the **"+"** icon on any line number in the diff to leave an inline comment.

2. Using Markdown Suggestion Blocks

Reviewers can propose exact code replacements directly in comment boxes:

```

```suggestion
 int reserved;
```

```

- When a suggestion is posted, GitHub renders an **"Apply suggestion"** button in the web UI.
- The author can click **"Commit suggestion"** to automatically apply the change directly from the browser!

3. Submitting the Review

Click the green **"Review changes"** button at the top right of the diff page: - **Comment:** Submit general feedback without explicit approval. - **Approve:** Mark the PR as approved for merge. - **Request changes:** Block merge until specified issues are resolved.

5.4 Monitoring Automated CI Checks & Actions on GitHub.com

At the bottom of the PR conversation tab, GitHub displays the status of all continuous integration workflows:

```
+-----+
|               CI CHECKS STATUS BOX               |
+-----+
|                                                     |
| [✓] All checks have passed (1 successful check)    [ Hide all checks ] |
|                                                     |
| [✓] build-and-test / Matrix (72/72 tests)          [ Details ] |
|                                                     |
| [ Merge pull request v ] Branch has no conflicts with base branch |
|                                                     |
+-----+
```

1. Click **"Details"** next to any failing check to inspect the live build logs, compiler warnings, or test failure output.
2. When all checks are green (), maintainers can click **"Merge pull request"** (or choose **"Squash and merge"** / **"Rebase and merge"** depending on upstream repository policy).

This pure single-theme playbook provides the cleanest, most reviewable pull-request submission possible on GitHub.